系统编程

Systems Programming

第二章　进程控制

授课时间　　第二周

本章学习目标：掌握进程生命周期管理

项目拆解

实现OSChatBox服务器守护进程

项目历程碑

服务器守护化
配置守护进程与日志文件

知识目标

- 理解进程状态及其转换原理
- 掌握进程标识符（PID）的获取与运用
- 剖析进程创建(fork())与终止（exit())机制
- 实现OSChatBox的守护化部署

能力目标

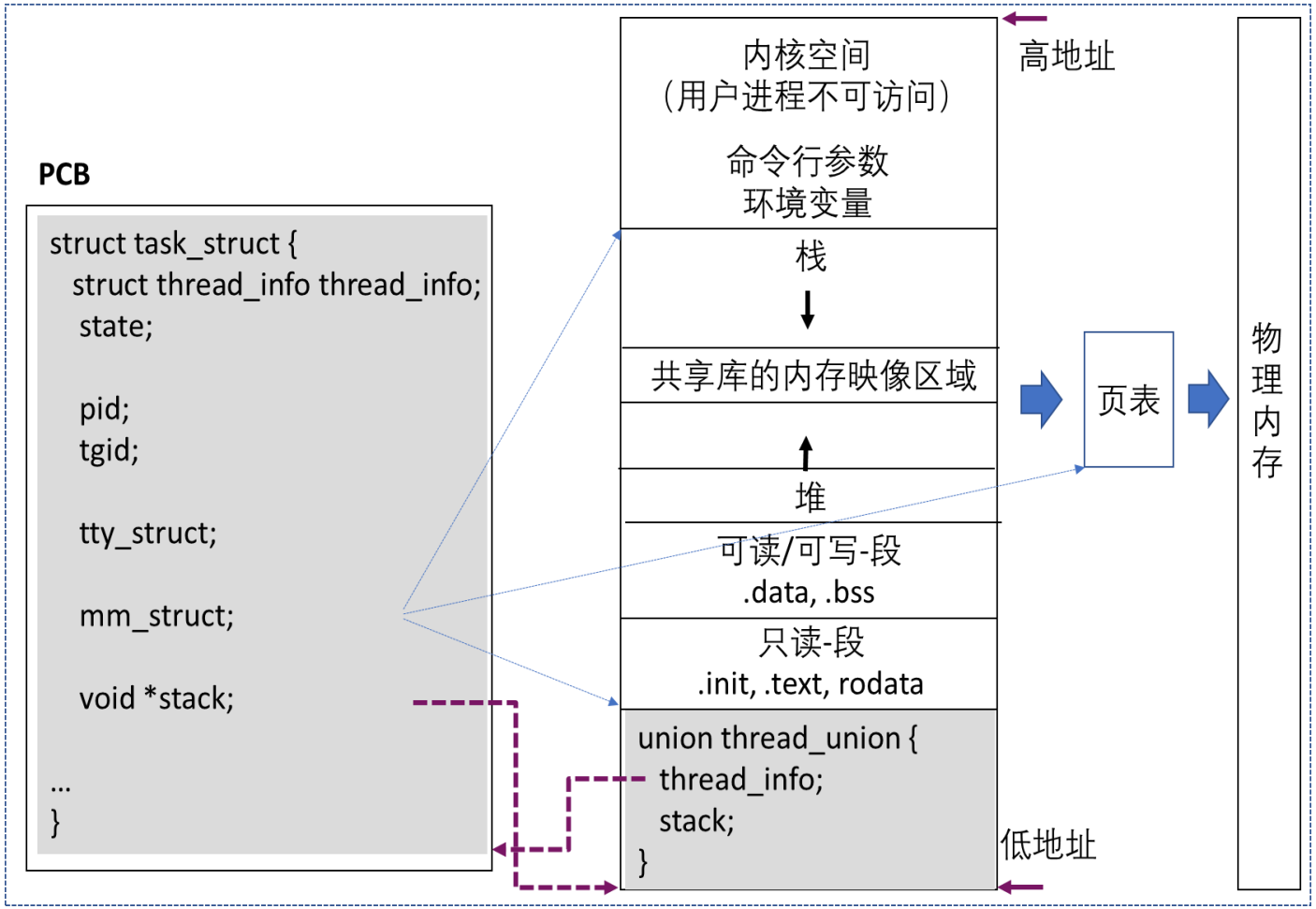
- 监控并管理进程生命周期
- 创建、终止和回收进程资源
- 编写健壮的守护进程
- 调试多进程并发问题

课程知识体系全景图

进程控制三大支柱

- 进程创建
- 进程终止
- 进程同步

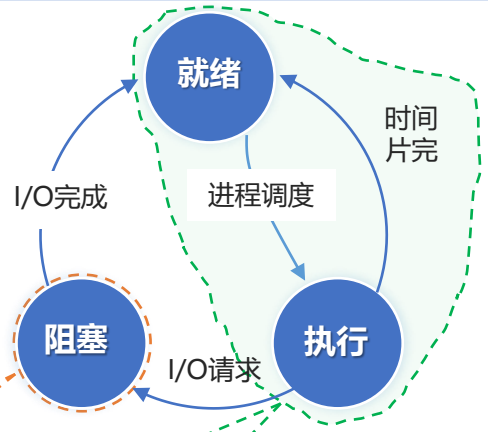
2.1 进程状态 – 内存映像



- 进程是程序执行时的一个实例
  - 内存映像
  - 页表
  - 物理内存
  - PCB
- 进程是系统资源分配的实体

2.1 进程状态

经典进程3状态模型（概念）

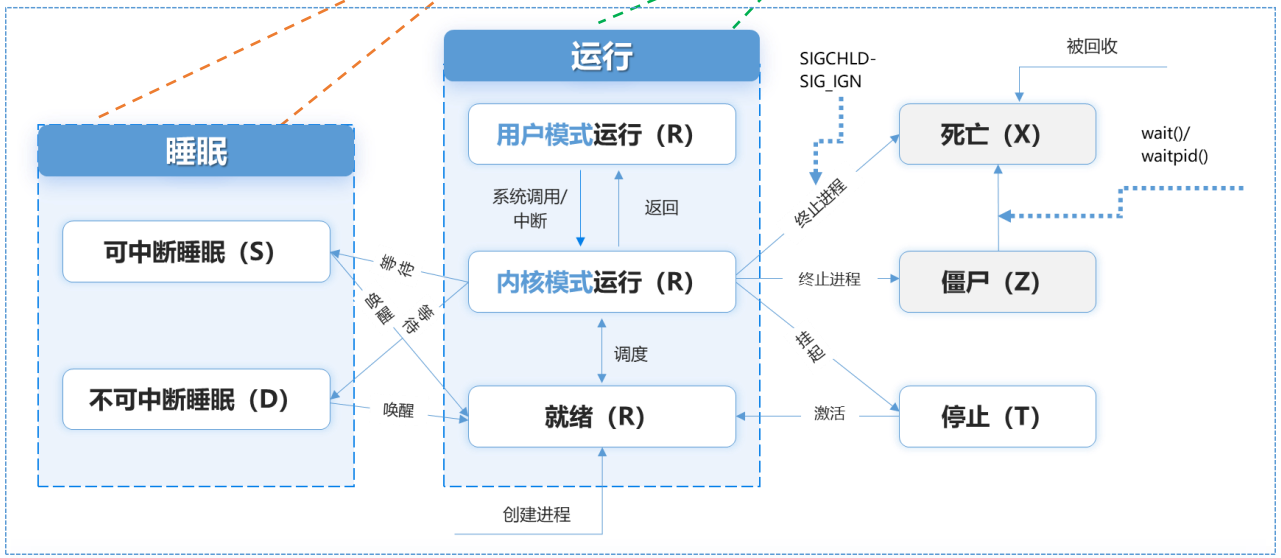



```
…
while (!condition_met) {
    set_current_state(TASK_INTERRUPTIBLE);
    schedule();
}
…
```

主动睡眠

让出CPU

Linux内核源码



openEuler进程状态（实现）

```
…
/* Used in tsk->state: */
#define TASK_RUNNING                    0x0000
#define TASK_INTERRUPTIBLE              0x0001
#define TASK_UNINTERRUPTIBLE            0x0002
#define __TASK_STOPPED                  0x0004
/* Used in tsk->exit_state: */
#define EXIT_DEAD                       0x0010
#define EXIT_ZOMBIE                     0x0020
…
```

/usr/src/kernels/4.19.90-2003.4.0.0036.oe1.aarch64/include/linux/sched.h

2.1 进程状态 –查看命令

常用命令

- ps

  显示瞬间进程的状态

- top / htop

  动态刷新系统负载与进程状态

- cat /proc/[PID]/status

  最全面的进程状态文件

ps常用参数

-l： 以长格式显示当前终端关联的进程

-u：按用户名和启动时间排序显示进程

-j： 以作业控制格式显示进程

-a： 查看系统所有进程

-x： 显示无控制终端的进程

-r： 显示运行中的进程

2.1 进程状态 – ps命令应用示例

```
1   $ps                  //显示当前终端中当前用户的进程
2   $ps –u user          //查看指定用户运行的所有进程
3   $ps –el              //详细列表显示所有进程
4   $ps aux              //显示所有进程的详细信息（BSD格式）

5   $ps -axl | grep PID; ps -axl | grep bash
6     F   UID      PID      PPID   PRI  NI  VSZ     RSS  WCHAN  STAT  TTY      TIME  COMMAND
7     0  1004 3749621 3749618  20   0  22724  3744   do_wai    Ss   pts/18    0:00    -bash
8   $ ps -axl | grep PID; ps -axl | grep kauditd
9     F  UID      PID      PPID   PRI  NI  VSZ     RSS  WCHAN  STAT  TTY      TIME  COMMAND
10    1   0        31        2     20   0   0        0         -       S     ?       0:26    [kauditd]
```

ps不能捕捉所有进程的“等待通道”函数

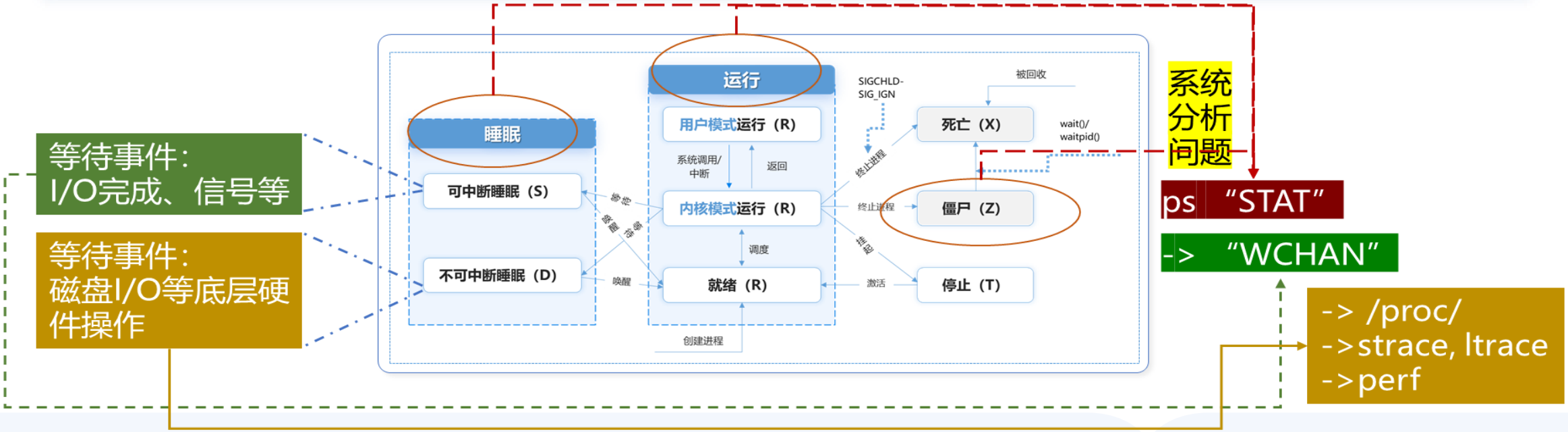
请完成课本P56 Q2.1-1

2.1 进程状态

```
1  $ sudo cat /proc/$(pgrep kauditd)/stack 2>/dev/null | head -5
2     [<0>] __switch_to+0xbc/0xfc
3     [<0>] kauditd_thread+0x1bc/0x310
4     [<0>] kthread+0x108/0x13c
5     [<0>] ret_from_fork+0x10/0x18
```

内核线程的WCHAN: “-”

- 内核调试工具：ftrace, perf
- 查看/proc/<pid>/stack



2.1 进程状态

```
1  $ps aux | more
2  USER  PID  %CPU  %MEM  VSZ  RSS  TTY  STAT START  TIME  COMMAND
3  root  1945  0.0  0.0  0  0  ?  I<  2023  0:00  [rpciod]
4  root  1948  0.0  0.0  18652  1644  ?  S<sl  2023  49:13  /sbin/auditd
5  root  1972  0.0  0.0  312324  1288  ?  Ssl  2023  849:34  /sbin/rngd -f
6  root  2110  0.0  0.0  13340  0  ?  Ss  2023  0:00  login -- root
7  root  2814  0.0  0.0  10228  8  tty1  Ss+  2023  0:00  -bash
```

- < (高优先级进程)
- N (低优先级进程)
- L (内存锁页)
- s (会话首进程)
- + (前台进程)
- l (多线程进程)

进程rngd状态显示为“Ssl”，请问该如何理解这一状态?

2.2 进程标识 - 获取进程/父进程ID

系统调用

```
#include <sys/types.h>
#include <unistd.h>
//获取进程标识
pid_t getpid(void);

//获取父进程标识
pid_t getppid(void);
```

应用例子

```
1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <unistd.h>
4  int main(int argc, char *argv[])
5  {
6      printf("My pid is %ld\n", (long)getpid());
7      printf("My parent' pid is %ld\n",(long)getppid());
8      sleep(10);
9  }
```

```
1  $ gcc -Wall -o getpid getpid.c
2  $ ./getpid
3  My pid is 180690
4  My parent' pid is 179855
```

```
5  $ ./getpid
6  My pid is 180703
7  My parent' pid is 179855
```

- **0号进程**
  - 命名：idle/swapper（具体系统实现有关）
  - 任务：创建2个内核进程（pid = 1 和 pid = 2）
  - 优先级别最低，仅当运行队列为空时被调度
- **1号进程**
  - 命名：systemd/init/upstart（具体系统实现有关）
  - 任务：启动和监控系统服务及用户会话
  - 所有用户态进程的祖先
- **2号进程**
  - 命名：kthreadd进程
  - 任务：创建和管理所有其他内核线程
  - 内核线程守护进程

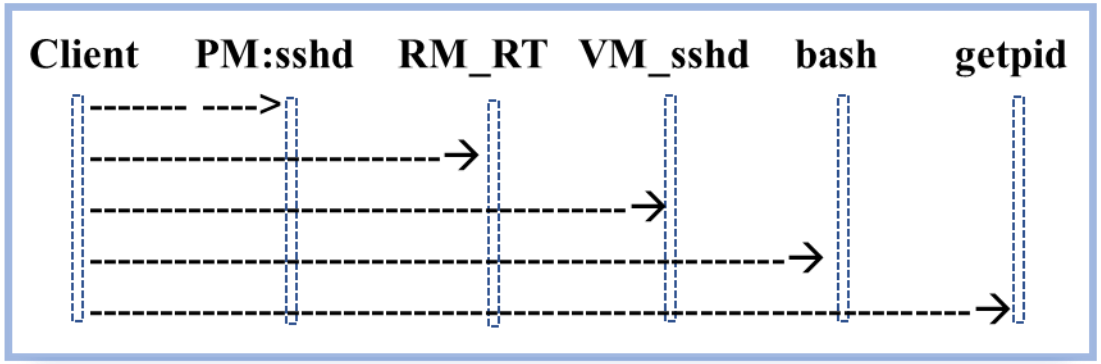
2.2 进程标识 – 获取进程层级（父子）关系

常用命令

- pstree
  以树状结构展示
- ps -f
  以树状结构展示进程，
  并展示每个进程的详细信息

例示

```
$ pstree
systemd─┬─NetworkManager─┬─dhclient
        |                └─2*[{NetworkManager}]
        ├─auditd───{auditd}
        ├─crond
        ......
        ├─rsyslogd───2*[{rsyslogd}]
        ├─sshd─┬─sshd───sshd───bash───pstree
        |      ├─2*[sshd───sshd───sftp-server]
        |      └─sshd───sshd───bash───getpid
        ├─systemd-logind
```



2.3 进程创建 – 2.3.1进程复制：fork()系统调用

```
#include <unistd.h>
pid_t  fork(void);
```

成功，返回值≥0

- 父进程返回子进程pid > 0
- 子进程返回0

失败，返回值<0

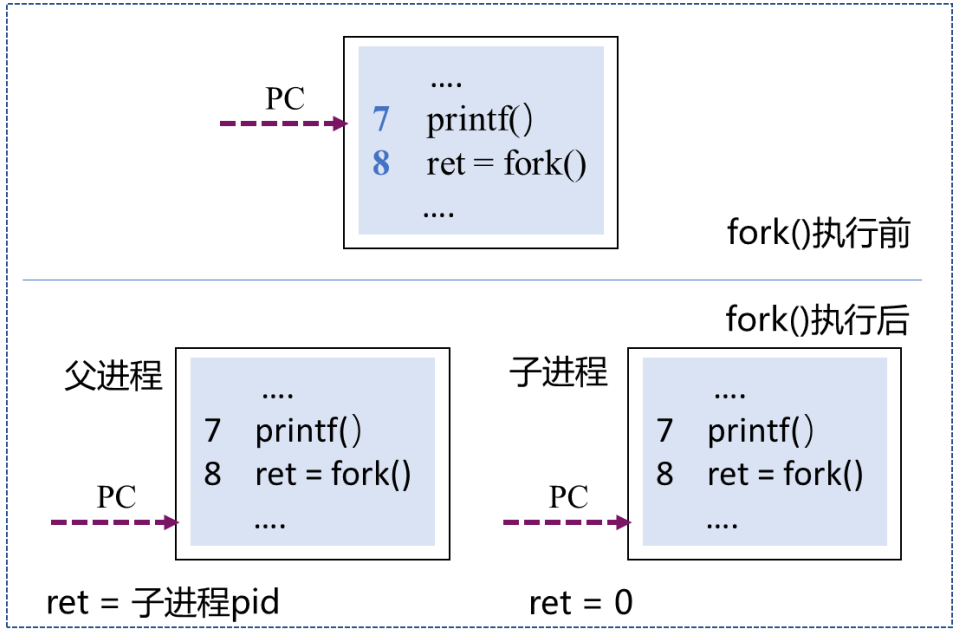
ERRNO	原因	建议处理措施
EAGAIN	进程数受限 - 用户或系统级上限	1. 等待并重试 2. 检查并调整RLIMIT_NPROC及相关配置
ENOMEM	内存分配失败 - 子进程创建受阻	1. 释放系统内存。 2. 检查是否存在内存泄漏。 3. 增加物理内存或交换空间
ENOSYS	平台不支持 - fork() 系统调用缺失	迁移至支持该调用的平台

```
1  #include <unistd.h>
2  #include <stdio.h>
3  pid_t fork_error_handler()
4  {
5      //to do...
6      return -1;
7  }
8  pid_t Fork(void)
9  {
10     pid_t ret = fork();
11     if (ret < 0) {
12         perror("Fail to fork in Fork()");
13         return fork_error_handler();
14     }
15     return ret;
16 }
```

Fork.c： 封装fork()的错误检查和处理

再次温馨提醒：系统调用要检查出错原因

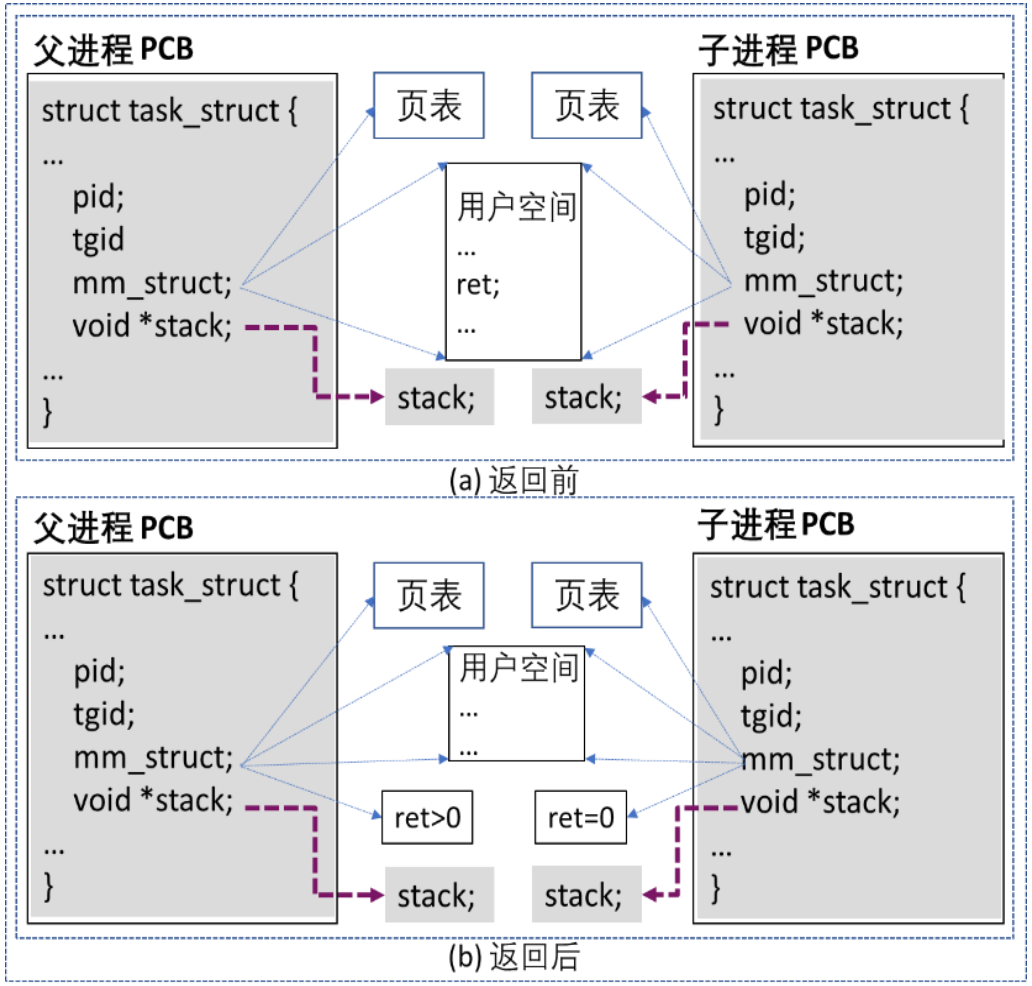
```
1  #include <stdio.h>
2  #include <unistd.h>
3  pid_t Fork(void);
4  int main(int argc, char *argv[])
5  {
6        pid_t ret;
7        printf( "Process with pid = %ld.\n",(long)getpid());
8        ret = Fork();
9        if (ret == 0){      //子进程
10             printf("Child pid: %ld\n",(long)getpid());
11       } else {            //父进程
12             printf("Parent pid: %ld\n",(long)getpid());
13       }
14 }
```



进程复制

2.3 进程创建 – 2.3.1进程复制：fork()系统调用

```
1  #include <stdio.h>
2  #include <unistd.h>
3  pid_t Fork(void);
4  int main(int argc, char *argv[])
5  {
6        pid_t ret;
7        printf( "Process with pid = %ld.\n",(long)getpid());
8        ret = Fork();
9        if (ret == 0){      //子进程
10             printf("Child pid: %ld\n",(long)getpid());
11       } else {            //父进程
12             printf("Parent pid: %ld\n",(long)getpid());
13        }
14 }
```



写时复制（Copy on Write, COW）

```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```

请问这个程序运行过程中

1）生成的进程数（含初始进程）？

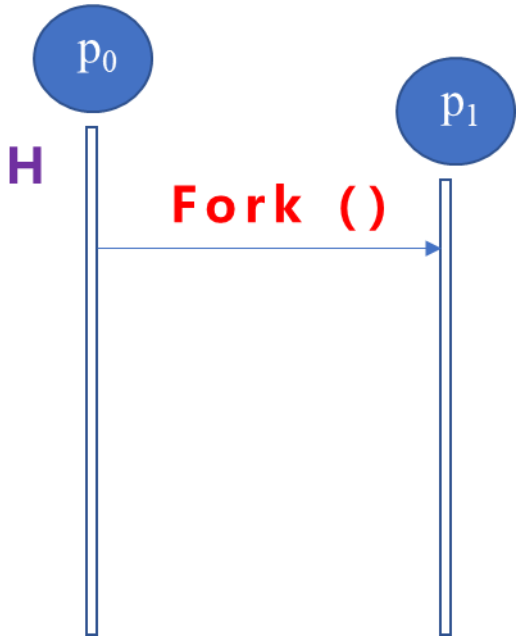
2）程序的预期输出结果？并解释输出顺序的成因

最好能图示关键创建点及流程

```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```

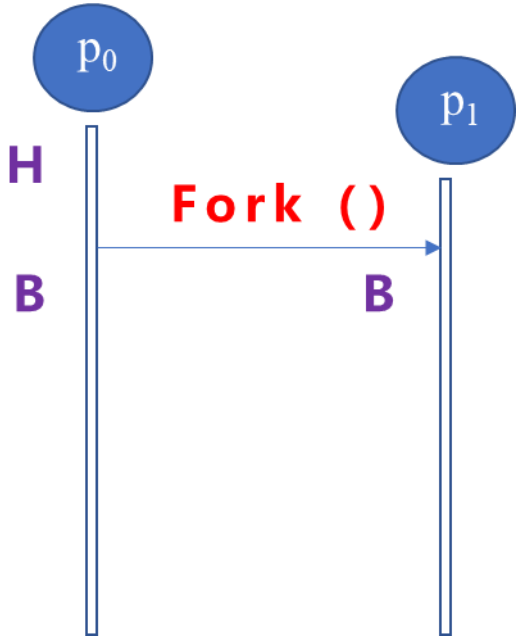
图示进程创建流程

```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```



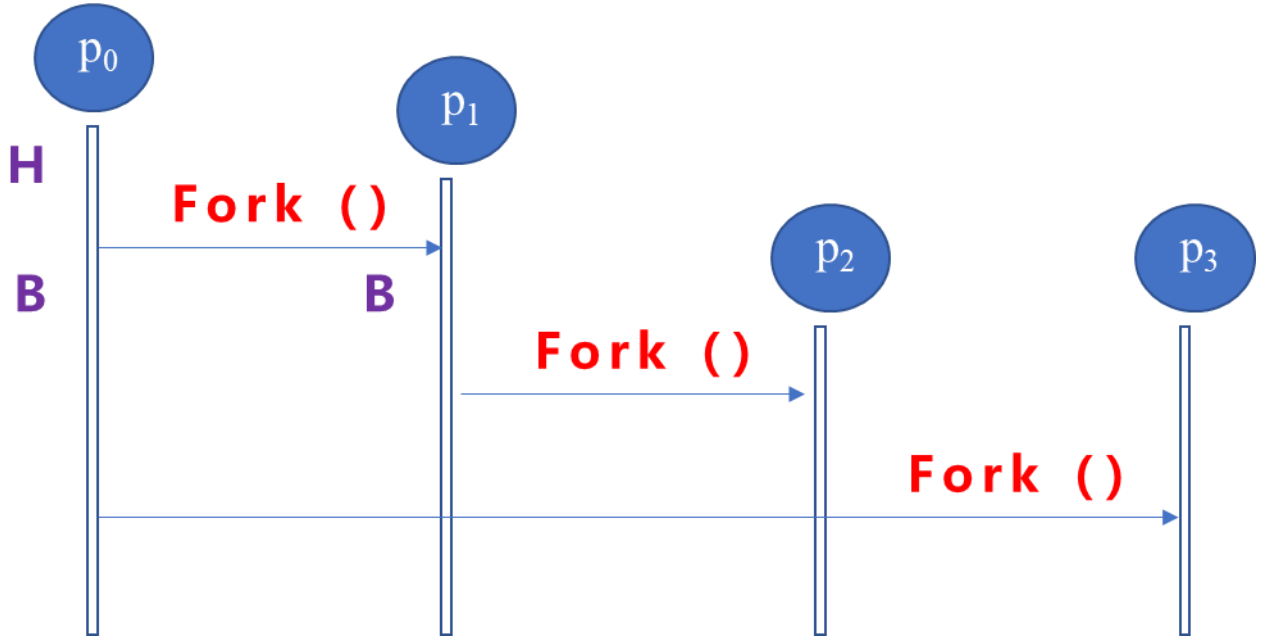
图示进程创建流程

```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```



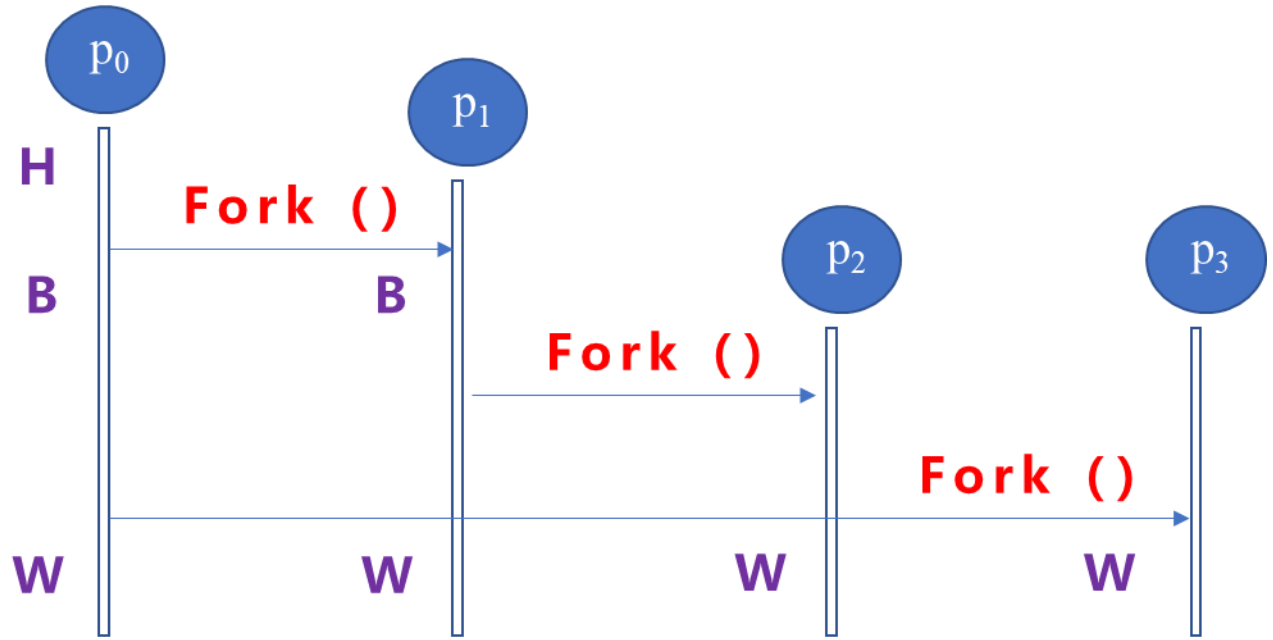
图示进程创建流程

```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```



图示进程创建流程

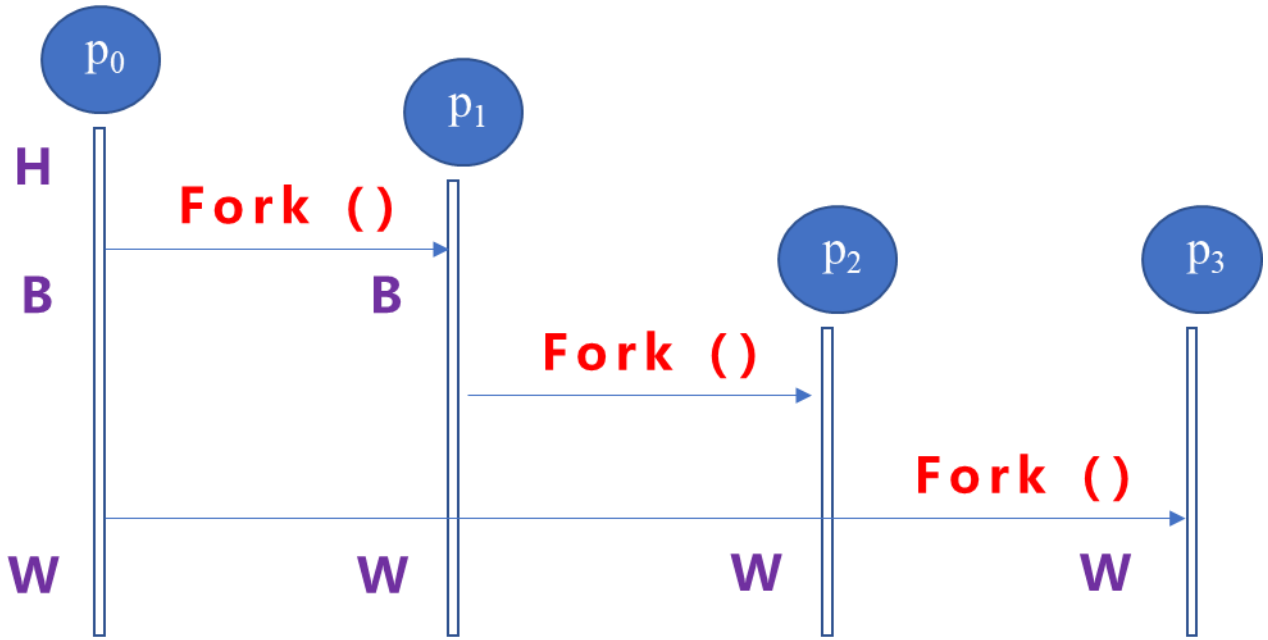
```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```



程序可能的输出结果及成因

A. HBBWWWW?　　B. HBWWBWW?

```
1   #include <stdio.h>
2   #include <unistd.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       printf("H\n");
7       Fork();
8       printf("B\n");
9       Fork();
10      printf("W\n");
11      return 0;
12  }
```



程序可能的输出结果及成因

A. HBBWWWW　　B. HBWWBWW

多个进程并发运行：输出交错，顺序不确定

```
1   #include <unistd.h>
2   #include <stdio.h>
3   pid_t Fork(void);
4   int main(int argc, char *argv[])
5   {
6       for (int i = 0; i < 2; i++){
7           Fork();
8       }
9       printf("*");
10      return 0;
11  }
```

请问这个程序运行过程中

1）生成的进程数（含初始进程）？

2）这段代码一共输出多少个字符‘*’

请图示关键创建点及流程
请上机验证自己的推断

```
1    pid_t Fork(void);
2    int main (int argc, char **argv)
3    {
4        int loop = 4;
5        while (loop > 0) {
6            if (Fork() == 0) break;
7            loop--;
8        }
9        printf ( "*");
10       return 0;
11   }
```

问题：

1）请解释这个程序的一次运行结果

```
[提示符]$./a.out
**[提示符]***
```

2）请修改代码，使得程序的每次执行结果都如下所示：

```
[提示符]$./a.out
*****
[提示符]$
```